Audited by CRED SHIELDS

CredShields

Smart Contract Audit

April 17, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of HeyElsa between April 1, 2026, and April 7, 2026. A retest was performed on April 13, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

HeyElsa

AICPA SOC

SCS Advocates

OWASP

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

Table of Contents

Table of Contents **2**

1. Executive Summary ——————————- **4**

State of Security 5

2. The Methodology ——————————- **6**

2.1 Preparation Phase 6

2.1.1 Scope 6

2.1.2 Documentation 6

2.1.3 Audit Goals 7

2.2 Retesting Phase 7

2.3 Vulnerability classification and severity 7

2.4 CredShields staff 9

3. Findings Summary ——————————- **10**

3.1 Findings Overview 10

3.1.1 Vulnerability Summary 10

5. Bug Reports —————————— **12**

Bug ID #H001 [Fixed] 12

Reward Theft via Withdraw-Relock — Global ve Numerator vs Eligible-Only Denominator 12

Bug ID #H002 [Fixed] 15

eligibleVeAtT Returns Zero for Historical Queries — Permanent Reward Destruction 15

Bug ID #H003 [Fixed] 17

_snapshotEligibleSupply() Backdates Mid-Week Deposits to the Epoch Start 17

Bug ID #M001 [Fixed] 18

tx.origin Validation in onlyUserOrWhitelist Will Break Under EIP-7702 18

Bug ID #M002 [Fixed] 19

Inconsistent lastPoint.blk Assignment in _checkpointEligible vs _fillPointHistory 19

Bug ID #M003 [Fixed] 21

depositFor Missing Blacklist Validation Allows Deposits to Restricted Addresses 21

Bug ID #L001 [Fixed] 22

claimPartial Allows Self-Inflicted Permanent Epoch Skipping 22

Bug ID #L002 [Fixed] 24

claimFor Allows Unsolicited Reward Claims Without User Consent 24

Bug ID #L003 [Fixed] 25

freezeEligibilityEnd Has No On-Chain Ordering Enforcement with RewardDistributor Initialization 25

Bug ID #L004 [Fixed] 27

No MINTIME Validation in Constructor 27

Bug ID #L005 [Fixed] 28

Missing events in important functions 28
6. The Disclosure ——————————- 29

1. Executive Summary ----------

HeyElsa engaged CredShields to perform a smart contract audit from April 1, 2026, to April 7, 2026. During this timeframe, 11 vulnerabilities were identified. **A retest was performed on April 13, 2026, and all the bugs have been addressed.**

During the audit, 3 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "HeyElsa" and should be prioritized for remediation.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Staking Contract V2	0	3	3	5	0	0	**11**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Staking Contract V2's scope during the testing window while abiding by the policies set forth by HeyElsa's team.

State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both HeyElsa's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at HeyElsa can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, HeyElsa can future-proof its security posture and protect its assets.

2. The Methodology ----------

HeyElsa engaged CredShields to perform a Staking Contract V2 audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from April 1, 2026, to April 7, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
• **Audited Scope:** https://github.com/HeyElsa/staking-contract-v2/tree/7799986c35cc01ae5452c383474af7db962bcd70 • **Retested Scope:** https://github.com/HeyElsa/staking-contract-v2/tree/ffa85f7940f865db1b1720d9368a6ba56e79b10b

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

HeyElsa is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat

agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities

can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 11 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Reward Theft via Withdraw-Relock — Global ve Numerator vs Eligible-Only Denominator	High	Business Logic (SC03-LogicErrors)	Fixed
eligibleVeAtT Returns Zero for Historical Queries — Permanent Reward Destruction	High	Permanent Fund Lock	Fixed
_snapshotEligibleSupply() Backdates Mid-Week Deposits to the Epoch Start	High	Inconsistent Accounting (SCWE-010)	Fixed
tx.origin Validation in onlyUserOrWhitelist Will Break Under EIP-7702	Medium	Incorrect Validation	Fixed
Inconsistent lastPoint.blk Assignment in _checkpointEligible vs _fillPointHistory	Medium	Inconsistent Accounting (SCWE-010)	Fixed
depositFor Missing Blacklist Validation Allows Deposits to Restricted Addresses	Medium	Access Control (SCWE-016)	Fixed
claimPartial Allows Self-Inflicted Permanent Epoch Skipping	Low	Lack of Input Validation (SC04-Lack Of Input Validation)	Fixed

claimFor Allows Unsolicited Reward Claims Without User Consent	Low	Access Control (SCWE-016)	Fixed
freezeEligibilityEnd Has No On-Chain Ordering Enforcement with RewardDistributor Initialization	Low	Missing Cross-Contract Validation	Fixed
No MINTIME Validation in Constructor	Low	Lack of Input Validation (SC04-Lack Of Input Validation)	Fixed
Missing events in important functions	Low	Missing Best Practices	Fixed

Table: Findings and Remediations

5. Bug Reports ----------

Bug ID #H001 [Fixed]

Reward Theft via Withdraw-Relock — Global ve Numerator vs Eligible-Only Denominator

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
High

Description
The reward distribution formula computes each user's share as userShare = (rewardsAllocated * userVe) / totalEligibleVe. The numerator userVe is derived from votingEscrow.balanceOfAtT(user, epochStartTime), which reads the global _userPointHistory that includes ve power from all locks regardless of eligibility status.

The denominator totalEligibleVe is derived from votingEscrow.eligibleVeAtT(epochStartTime), which reads the eligible-only _eligiblePointHistory that only counts ve from users who locked during the eligibility window. This creates a population mismatch between the numerator and denominator.

The claim gate check wasSeasonEligible(user), a permanent flag set during _createLock that is never cleared. When a user withdraws, only seasonEligible is cleared to false while wasSeasonEligible remains true. This means a user who was once eligible permanently retains the right to call claim().

The exploit works as follows:

1. Alice creates a lock during the eligibility window, which sets both seasonEligible and wasSeasonEligible to true.
2. After Alice's lock expires she calls withdraw(), which clears seasonEligible to false but leaves wasSeasonEligible as true.
3. The eligibility window then closes.

4. Alice calls createLock again with a massive amount such as 100,000 ELSA for 4 years. Since the eligibility window is closed, the new lock is not added to _eligiblePointHistory but it is recorded in the global _userPointHistory.
5. When the epoch is checkpointed, totalEligibleVe stores only the legitimate eligible users' ve such as Bob's 500 veELSA.
6. When Alice calls claim(), wasSeasonEligible passes the gate. balanceOfAtT returns Alice's global ve of approximately 80,000 veELSA from her re-lock.
7. The formula then computes userShare = rewardsAllocated * 80,000 / 500, yielding 160 times the epoch's intended rewards.
8. The rewardsMax cap limits total protocol-wide extraction, but Alice claims first and takes 100% of the budget while Bob receives nothing.

Affected Code

- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L648
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L684
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L389
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L614
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L799-L813

Impacts

An attacker can directly steal rewards from all other eligible users. Because the numerator draws from global ve history while the denominator only includes eligible ve, the attacker's computed share can exceed 100% of an epoch's allocated rewards.

The rewardsMax cap limits the total amount extractable from the contract across all epochs, but it does not prevent the attacker from claiming first and draining the entire epoch budget, leaving legitimate eligible users with zero rewards for that epoch.

The attack is repeatable across every epoch in the season, and the amplification factor is proportional to the re-lock amount — a 100x re-lock yields approximately a 100x theft multiplier against the eligible denominator. All eligible users who have not yet claimed are affected.

Remediation

Replace the global balanceOfAtT call in _getUserVeAtEpochCached with an eligible-only user balance query so that the numerator and denominator are drawn from the same population.
The recommended approach is to add an eligibleBalanceOfAtT(user, t) function to VotingEscrow that returns a user's ve only if they were seasonEligible at the relevant time and returns 0

otherwise. An alternative approach is to clear wasSeasonEligible in _createLock when a user re-locks after the eligibility window has closed, preventing them from passing the claim gate entirely.

Retest

This issue has been fixed by resetting wasSeasonEligible to false during ineligible re-locks in _createLock() to prevent reward theft attacks.

Bug ID #H002 [Fixed]

eligibleVeAtT Returns Zero for Historical Queries — Permanent Reward Destruction

Vulnerability Type
Permanent Fund Lock

Severity
High

Description
VotingEscrow.eligibleVeAtT() always starts its slope-walk from the latest eligible point (_eligiblePointHistory[eligiblePointEpoch]). The internal function _eligibleSupplyAt returns 0 immediately if the query timestamp t is earlier than the latest point's timestamp:

```
if (t < lastPoint.ts) {
  return 0;
}
```

Unlike the global _supplyAt function which uses a binary search through _pointHistory to find the correct starting point for any historical timestamp, eligibleVeAtT has no binary search — it reads only the single latest point.

This means that whenever an eligible user performs any lock-modifying action (createLock, increaseUnlockTime, withdraw), the eligible point epoch advances to the current block timestamp. Any subsequent eligibleVeAtT query for a timestamp before this new point returns 0.

When RewardDistributor.checkpointEpoch(epochId), it stores votingEscrow.eligibleVeAtT(epochStartTime) as totalEligibleVe. If this returns 0, the epoch's totalEligibleVe is permanently set to 0. At claim time (_claimRange), epochs with totalEligibleVe == 0 are skipped via continue — the allocated rewards for that epoch sit in the contract forever.

Since checkpointEpoch sets data.checkpointed = true, the epoch cannot be re-checkpointed (reverts with EpochAlreadyCheckpointed). The reward destruction is permanent.

Attack sequence (griefing):

1. Alice locks at season start (Week 0). Eligible point is at Week 0.

2. No one checkpoints epochs 0-3 during Weeks 1-3.
3. At Week 4, Alice calls increaseUnlockTime. Eligible point advances to Week 4.
4. Anyone checkpoints epochs 0-3. For each: eligibleVeAtT(epochStart) → latest point ts = Week 4 → epochStart < Week 4 → returns 0.
5. All 4 epochs have totalEligibleVe = 0 with nonzero rewardsAllocated. ~1,162 ELSA permanently locked.
6. This also occurs organically without malice: any eligible user locking during Week 2 while epoch 1 is uncheckpointed destroys epoch 1's rewards.

Affected Code

- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L200-L205
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L254-L256
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L389
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L398-L400

Impacts

Allocated ELSA tokens become permanently locked in the RewardDistributor contract with no recovery mechanism. At approximately 697K ELSA per epoch (REWARDS_MAX / 43 epochs), destroying even a few epochs causes significant protocol-wide loss.

A single griefing transaction such as increaseUnlockTime can destroy all uncheckpointed epochs simultaneously by advancing the eligible point past their start times.

The bug can also occur without malice through normal user activity — any eligible user locking or extending after an epoch's start time but before that epoch is checkpointed destroys it — making this a systemic risk rather than a purely adversarial one. The damage is irreversible because the checkpointed flag prevents re-checkpointing and there is no admin function to correct stored epoch data.

Remediation

Implement a binary search in eligibleVeAtT to find the correct historical starting point, mirroring the pattern used in the global _findBlockEpoch and _supplyAt functions. This ensures that queries for any past timestamp find the correct eligible point to begin the slope-walk from

Retest

This issue has been fixed by implementing a gas-efficient binary search in eligibleVeAtT() to accurately locate _eligiblePointHistory entries for historical queries.

Bug ID #H003 [Fixed]

_snapshotEligibleSupply() Backdates Mid-Week Deposits to the Epoch Start

Vulnerability Type
Inconsistent Accounting (SCWE-010)

Severity
High

Description
_snapshotEligibleSupply() unconditionally overwrites _eligibleSupplySnapshots[weekStart] on every eligible user action. The eligibleSupply counter is updated before the snapshot call, and the snapshot rounds block.timestamp down to the week start so a mid-week deposit on Wednesday writes the post-deposit value into the Monday (epoch-start) slot. This retroactively backdates the deposit to the beginning of the epoch. RewardDistributor.checkpointEpoch() then reads this inflated value via eligibleSupplyAtT(epochStartTime) as the eligibleTVL denominator, distorting TVL-scaled emission factor calculations and per-user reward shares. Additionally, each subsequent action within the same week overwrites the snapshot again, meaning the final snapshot reflects the last action's state rather than the epoch-boundary state.

Affected Code
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L917-L923

Impacts
The TVL-scaled emissionFactor (eligibleTVL >= tvlTarget ? PRECISION : (eligibleTVL * PRECISION) / tvlTarget) uses an inaccurate eligibleTVL, potentially triggering full reward emission for an epoch where the TVL target was not actually met at the epoch boundary. Per-user reward shares are distorted: users staked since the true epoch start are diluted, while late depositors benefit from having their principal counted for the full epoch.

Remediation
It's recommended to Snapshot before updating eligibleSupply. If it's doing a deposit in mid-week then add in next week.

Retest
Fixed in cfc58663b36ba3af2ca86fea014a0d17b458bbbe

Bug ID #M001 [Fixed]

tx.origin Validation in onlyUserOrWhitelist Will Break Under EIP-7702

Vulnerability Type
Incorrect Validation

Severity
Medium

Description
The onlyUserOrWhitelist modifier uses msg.sender != tx.origin to determine if the caller is a smart contract, assuming that tx.origin == msg.sender always holds true for EOA callers. EIP-7702 (active on Ethereum mainnet since the Pectra upgrade) allows an EOA to set a delegation designator pointing to a contract's code. When a delegated EOA calls the contract, msg.sender is the EOA itself but it now has code attached, and depending on the call path tx.origin may differ from msg.sender. More critically, when another contract invokes the delegated EOA's logic on behalf of the user, msg.sender != tx.origin evaluates to true, causing the modifier to revert even though the caller is a legitimate EOA with delegated code — not a malicious smart contract.

Affected Code
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L104-L109

Impacts
Users who adopt EIP-7702 delegation for their smart EOAs will be able to call createLock(), increaseAmount(), increaseUnlockTime(), and increaseAmountAndTime() unless the delegated code address is manually whitelisted. As EIP-7702 adoption grows, an increasing number of legitimate users will be unable to participate in staking.

Remediation
It is recommended to replace the tx.origin check with a code-size check to reliably detect contract callers.

Retest
This issue has been fixed by updating the modifier to accept only onlyNotBlacklisted() users.

Bug ID #M002 [Fixed]

Inconsistent lastPoint.blk Assignment in _checkpointEligible vs _fillPointHistory

Vulnerability Type
Inconsistent Accounting (SCWE-010)

Severity
Medium

Description
The global _fillPointHistory function correctly interpolates block numbers for historical intermediate weekly points using blockSlope, approximating the actual block at each past timestamp. However, the _checkpointEligible function assigns block.number to all intermediate historical points regardless of their timestamp. When multiple weeks of history are filled in a single transaction, all intermediate _eligiblePointHistory entries are stamped with the current block number even though they represent timestamps potentially weeks apart. This creates a divergence where _pointHistory has properly interpolated block numbers but _eligiblePointHistory has identical, incorrect block numbers for all backfilled entries.

Affected Code
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L878

Impacts
The _eligiblePointHistory array stores incorrect block numbers for historical points. Multiple distinct weekly timestamps map to the same block number, violating the invariant that point.blk should approximate the block at point.ts. Currently, eligible supply queries use timestamp-based lookup so reward calculations are unaffected. However, any future on-chain or off-chain functionality performing block number based binary search over _eligiblePointHistory would return inaccurate results.

Remediation
It is suggested to apply the same interpolation logic used in _fillPointHistory to _checkpointEligible.

Retest

This issue has been fixed by applying blockSlope interpolation to _checkpointEligible to accurately calculate intermediate block numbers, ensuring consistency with _pointHistory.

Bug ID #M003 [Fixed]

depositFor Missing Blacklist Validation Allows Deposits to Restricted Addresses

Vulnerability Type
Access Control (SCWE-016)

Severity
Medium

Description
The depositFor() function allows any external caller to deposit tokens into an existing lock on behalf of another address. Unlike createLock, increaseAmount, and increaseUnlockTime which are gated by the onlyUserOrWhitelist modifier depositFor performs no validation on whether the target address is blacklisted or sanctioned.

Affected Code

- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L317-L329

Impacts
A sanctioned or malicious address that has been identified and should be restricted from further protocol participation can still have its position strengthened through third-party deposits.

Remediation
Introduce a blacklist mapping and enforce it in depositFor and other relevant entry points

Retest
Fixed in cfc58663b36ba3af2ca86fea014a0d17b458bbbe

Bug ID #L001 [Fixed]

claimPartial Allows Self-Inflicted Permanent Epoch Skipping

Vulnerability Type
Lack of Input Validation (SC04-Lack Of Input Validation)

Severity
Low

Description
The claimPartial function allows a user to specify a fromEpoch that is greater than their current userLastClaimedEpoch. The validation only prevents fromEpoch from going backwards below the last claimed epoch but does not prevent it from jumping forwards. When the user supplies a fromEpoch greater than userLastClaimedEpoch, the _claimRange function initializes lastProcessedEpoch = fromEpoch and begins iteration from there. After the loop completes, userLastClaimedEpoch[user] is updated to lastProcessedEpoch, permanently advancing the pointer past all intermediate epochs. Those skipped epochs can never be claimed because any subsequent call to claimPartial will clamp fromEpoch upward to the now-advanced userLastClaimedEpoch.

This is self-inflicted only because claimPartial uses msg.sender and cannot be called on behalf of another user. However it represents a significant footgun with no recovery path once executed.

Affected Code
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L474-L477
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L631
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L666

Impacts
A user who calls claimPartial with a fromEpoch ahead of their current pointer permanently forfeits all rewards for the skipped epochs. The loss can be substantial depending on how many reward-bearing epochs are skipped. The forfeited rewards remain locked in the contract and are not redistributable to other users. The risk is limited to self-infliction through user error or interaction with a malicious frontend and is not externally exploitable by other users.

Remediation

Reject any fromEpoch that would skip unclaimed epochs by requiring it to equal the user's current claim pointer. This prevents accidental gaps while still allowing the user to specify a narrower upper bound via toEpoch.

Retest

This issue has been fixed by overriding the fromEpoch parameter in claimPartial() to prevent users from skipping unclaimed epochs

Bug ID #L002 [Fixed]

claimFor Allows Unsolicited Reward Claims Without User Consent

Vulnerability Type
Access Control (SCWE-016)

Severity
Low

Description
The claimFor function is callable by any address without any approval or consent mechanism from the target user. The internal _claim function sends rewards to the target user and advances their userLastClaimedEpoch pointer. While the funds are transferred to the correct user and not to the caller, the target user has no control over the timing of their reward receipt. Any third party can trigger the claim at any moment once claiming is enabled.

Affected Code
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L446-L449
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L612-L627

Impacts
In many jurisdictions token receipt triggers a taxable event, and a third party can force this timing on a user without consent. The user's userLastClaimedEpoch is advanced, removing their ability to batch claims or time them strategically for tax optimization or other purposes. No funds are lost since rewards always go to the correct user, but the user loses sovereignty over their claim timing.

Remediation
Add an opt-in approval mechanism so that only the user themselves or pre-approved addresses can trigger claims on their behalf.

Retest
This issue has been fixed by introducing an opt-out mechanism allowing users to block third parties from executing claims on their behalf via claimFor().

Bug ID #L003 [Fixed]

freezeEligibilityEnd Has No On-Chain Ordering Enforcement with RewardDistributor Initialization

Vulnerability Type
Missing Cross-Contract Validation

Severity
Low

Description
The freezeEligibilityEnd function permanently freezes the eligibilityEnd timestamp by setting eligibilityEndFrozen = true, preventing any further calls to setEligibilityEnd. The NatSpec comment states it "should be called after the RewardDistributor is initialized," but there is no on-chain enforcement of this ordering constraint. The only validation is that eligibilityEnd is nonzero. There is no check that the RewardDistributor contract has been initialized or that its eligibilityEnd parameter matches the VotingEscrow's value.

If the owner calls freezeEligibilityEnd before RewardDistributor.initialize(), the eligibility end is permanently locked. If the RewardDistributor is subsequently initialized with a different eligibilityEnd_ value, the two contracts would operate with mismatched eligibility windows. The VotingEscrow would gate user lock actions against one timestamp while the RewardDistributor would calculate reward allocations against another.

Affected Code
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L146-L150
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/RewardDistributor.sol#L154

Impacts
If the deployment script calls these functions in the wrong order or with mismatched parameters, the eligibility window would be inconsistent between the two contracts. This is an operational risk that requires admin error during deployment and is not exploitable by regular users. Once frozen, there is no recovery path and the contract would need to be redeployed.

Remediation
Add a cross-contract consistency check before allowing the freeze. The VotingEscrow should verify that the RewardDistributor is initialized and that both contracts agree on the eligibilityEnd value.

Retest

This issue has been fixed by adding a cross-contract consistency check.

Bug ID #L004 [Fixed]

No MINTIME Validation in Constructor

Vulnerability Type
Lack of Input Validation (SC04-Lack Of Input Validation)

Severity
Low

Description
The VotingEscrow constructor accepts minTime as a parameter and assigns it directly to the immutable variable MINTIME without any bounds validation. If MINTIME is set to 0, users can create extremely short locks that expire within the same season. This significantly lowers the barrier for the Bug #1 withdraw-relock attack because the attacker's original eligible lock can expire mid-season, allowing withdrawal and re-lock while rewards are still accruing. If MINTIME is set to a value greater than MAXTIME (4 years), no locks can ever be created because the unlock time check in _createLock would always revert with UnlockTimeTooShort. Since MINTIME is declared as immutable, a misconfigured deployment cannot be corrected and would require redeploying the entire contract.

Affected Code
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L121
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L739-L742

Impacts
A MINTIME of 0 enables very short locks and lowers the cost of executing the Bug #1 withdraw-relock attack by allowing mid-season lock expiry without requiring admin intervention via unlock(). A MINTIME greater than MAXTIME permanently bricks all lock creation, rendering the entire protocol unusable. Since the variable is immutable, there is no recovery without full contract redeployment.

Remediation
Add validation to ensure MINTIME falls within a valid range.

Retest
This issue has been fixed by adding a validation on MINTIME.

Bug ID #L005 [Fixed]

Missing events in important functions

Vulnerability Type
Missing Best Practices

Severity
Low

Description
Events are inheritable members of contracts. When you call them, they cause the arguments to be stored in the transaction's log—a special data structure in the blockchain. These logs are associated with the address of the contract which can then be used by developers and auditors to keep track of the transactions.

The contract was found to be missing these events on certain critical functions which would make it difficult or impossible to track these transactions off-chain.

Affected Code
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L146-L150
- https://github.com/HeyElsa/staking-contract-v2/blob/7799986c35cc01ae5452c383474af7db962bcd70/src/VotingEscrow.sol#L152-L162

Impacts
Events are used to track the transactions off-chain and missing these events on critical functions makes it difficult to audit these logs if they're needed at a later stage.

Remediation
Consider emitting events for important functions to keep track of them.

Retest
This issue has been fixed by emitting events at important functions.

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

SCS Advocates

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com